

TECHNICAL_REQUIREMENTS_DOCUMENT.md

EduMentor AI

Technical Requirements Document (TRD)

Version: 2.0

Status: MVP

Overview

This document defines the technical architecture, infrastructure, development standards, backend services, API design, security requirements, and deployment strategy for EduMentor AI.

The goal is to create a scalable, secure, and maintainable AI-powered learning platform.

System Architecture

High-Level Architecture



▼
Groq API
(Llama 3.3 70B)

Technology Stack

Layer	Technology
Frontend	Next.js 15
Language	TypeScript
Styling	Tailwind CSS
Components	shadcn/ui
Icons	Lucide React
Charts	Recharts
Backend	Next.js Route Handlers
Database	PostgreSQL
Auth	Supabase Auth
Vector DB	pgvector
AI Model	Llama 3.3 70B Versatile
AI Provider	Groq
Embeddings	Transformers.js
Embedding Model	all-MiniLM-L6-v2
Hosting	Render

Project Structure

```
src/  
|  
├─ app/  
|   ├─ (auth)/  
|   ├─ dashboard/  
|   ├─ api/  
|   └─ onboarding/
```

```
|
| components/
| services/
| lib/
| hooks/
| types/
| utils/
| middleware/
```

Frontend Requirements

Framework

Use:

```
Next.js 15
App Router
TypeScript
```

Styling

Use:

```
Tailwind CSS
```

Component library:

```
shadcn/ui
```

State Management

For MVP:

React Context
React Hooks

No Redux required.

Backend Requirements

Architecture

Backend will use:

Next.js Route Handlers

No separate Express server.

API Design

All AI requests must go through backend routes.

Required architecture:

Browser
↓
API Route
↓
Groq

Forbidden:

Browser
↓
Groq

Core Services

Authentication Service

File:

```
services/auth.service.ts
```

Responsibilities:

- Registration
 - Login
 - Logout
 - Session Validation
-

Conversation Service

File:

```
services/conversation.service.ts
```

Responsibilities:

- Create Sessions
 - Rename Sessions
 - Delete Sessions
 - Load Sessions
-

Chat Service

File:

```
services/chat.service.ts
```

Responsibilities:

- Build Prompt
- Load Context
- Call Groq

- Store Responses
-

Quiz Service

File:

```
services/quiz.service.ts
```

Responsibilities:

- Generate Quiz
 - Evaluate Quiz
 - Store Results
-

Progress Service

File:

```
services/progress.service.ts
```

Responsibilities:

- Calculate Mastery
 - Update Progress
 - Generate Analytics
-

Rate Limit Service

File:

```
services/rate-limit.service.ts
```

Responsibilities:

- Track Usage
 - Validate Limits
 - Block Abuse
-

Database Requirements

Provider:

Supabase

Database:

PostgreSQL

Required Extension

```
CREATE EXTENSION IF NOT EXISTS vector;
```

Tables

Required tables:

```
profiles
user_preferences
conversations
messages
quizzes
quiz_results
progress
learning_analytics
api_usage
```

knowledge_chunks

Authentication Requirements

Provider:

Supabase Auth

Authentication Flow

```
Register
↓
Login
↓
Session
↓
Protected Routes
```

Route Protection

Protect:

```
/dashboard
/dashboard/learn
/dashboard/quiz
/dashboard/progress
/profile
/settings
```

AI Requirements

Provider:

Groq

Model:

Llama 3.3 70B Versatile

AI Tutor Requirements

The tutor must:

- Explain Python concepts
- Adapt to user level
- Use conversation history
- Maintain educational focus

Prompt Construction

Structure:

System Prompt

+

Conversation History

+

Current User Message

Context Window Strategy

To reduce cost and latency.

Send:

Last 15 Messages

plus:

Current User Message

Future:

Top 5 Knowledge Chunks

Embedding Requirements

Provider:

Transformers.js

Model:

all-MiniLM-L6-v2

Output:

384 Dimensions

Adaptive Learning Engine

Quiz generation must use mastery score.

Difficulty Rules

Mastery Score	Difficulty
0 - 40	Beginner

Mastery Score	Difficulty
41 - 70	Intermediate
71 - 100	Advanced

Quiz Generation Flow

```
graph TD; A[Load Progress] --> B[Read Mastery]; B --> C[Determine Difficulty]; C --> D[Generate Quiz];
```

Rate Limiting Requirements

Purpose:

- Prevent Abuse
- Protect API Usage
- Control Costs

Chat Limits

100 Requests / Day

per user.

Quiz Limits

50 Generations / Day

per user.

API Routes

Authentication

`/api/auth/register`

`/api/auth/login`

`/api/auth/logout`

Chat

`/api/chat`

Method:

`POST`

Quiz

`/api/quiz/generate`

`/api/quiz/submit`

`/api/quiz/history`

Progress

`/api/progress`

Security Requirements

Environment Variables

Public:

```
NEXT_PUBLIC_SUPABASE_URL=  
NEXT_PUBLIC_SUPABASE_ANON_KEY=
```

Private:

```
SUPABASE_SERVICE_ROLE_KEY=  
GROQ_API_KEY=
```

Never expose private keys.

Row Level Security

Enable RLS on:

```
profiles  
user_preferences  
conversations  
messages  
quizzes  
quiz_results  
progress  
learning_analytics
```

api_usage

Data Access Rules

Users may only access:

- Their sessions
- Their messages
- Their quizzes
- Their analytics

Never another user's data.

Error Handling

API Errors

Return:

```
{
  "success": false,
  "message": "Request failed"
}
```

AI Errors

Return safe fallback messages.

Never expose:

- API keys
 - Stack traces
 - Internal errors
-

Logging Requirements

Allowed:

API Errors
Database Errors
Performance Metrics

Forbidden:

Passwords
JWT Tokens
API Keys
Private Messages

Performance Requirements

Dashboard

Load Time:

< 2 Seconds

Chat Responses

Response Time:

< 3 Seconds

Quiz Generation

Response Time:

< 5 Seconds

Deployment Requirements

Provider:

Render

Continuous Deployment

Deployment starts on Day 1.

Every push to:

main

must automatically deploy.

Environment Configuration

Render Environment Variables:

```
NEXT_PUBLIC_SUPABASE_URL=  
NEXT_PUBLIC_SUPABASE_ANON_KEY=  
SUPABASE_SERVICE_ROLE_KEY=  
GROQ_API_KEY=
```

Monitoring Checklist

Verify:

- Authentication
 - Database Connectivity
 - Groq Connectivity
 - API Routes
 - Deployment Status
-

MVP Acceptance Criteria

The system is considered complete when:

1. User can register.
 2. User can login.
 3. User can complete onboarding.
 4. User can create learning sessions.
 5. User can chat with the AI tutor.
 6. User can generate adaptive quizzes.
 7. User can track mastery progress.
 8. User can resume learning sessions.
 9. RLS is enforced.
 10. Groq API key remains private.
-

Future Technical Roadmap

Not part of MVP.

- RAG Knowledge Base
 - File Upload Support
 - Multi-Language Learning
 - Voice Tutor
 - Code Execution Sandbox
 - Mobile Application
 - AI Roadmap Generator
 - Teacher Dashboard
 - Parent Dashboard
-

Final Technical Objective

Build a secure, scalable AI tutoring platform that provides:

- Personalized Learning
- Adaptive Assessments
- Persistent Conversations
- Progress Tracking
- Secure User Data

while maintaining a lightweight architecture suitable for rapid MVP development and hackathon deployment.